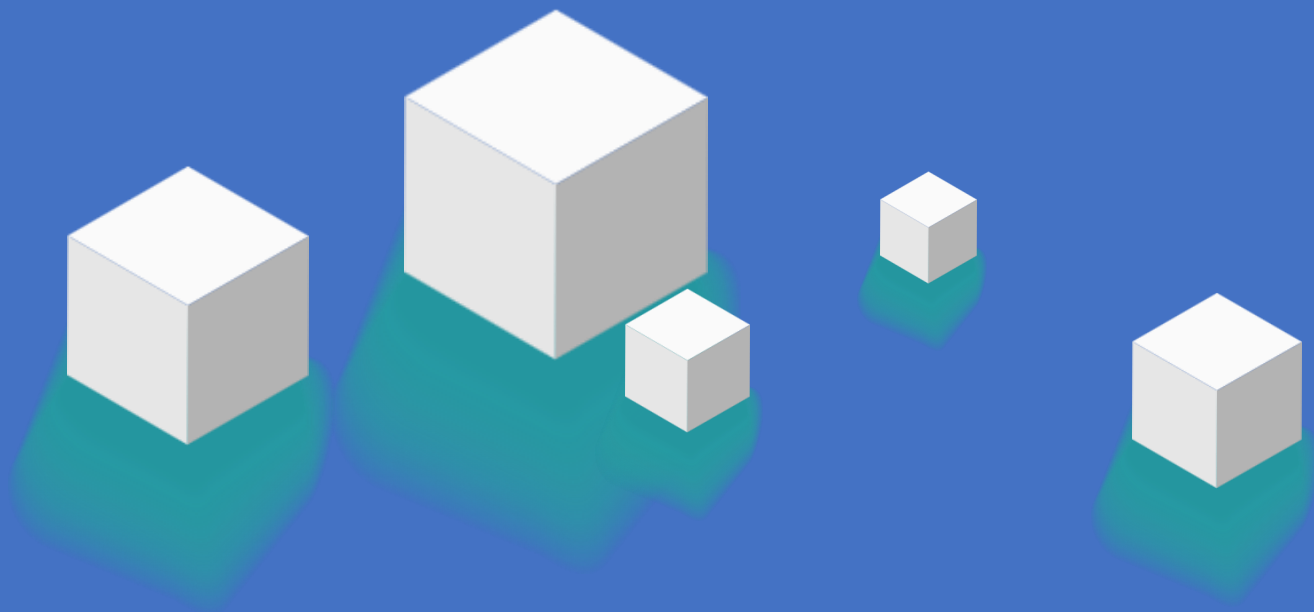


HuggingFace生态实战： 从模型应用到高效微调



>> 今天的学习目标

HuggingFace生态实战

- 什么是HuggingFace
- HuggingFace的核心
- Pipeline与模型应用
- 全量微调
- CASE：垃圾邮件分类器

什么是HuggingFace

Thinking: 大家可能听说过 TensorFlow 或 PyTorch, 那什么是HuggingFace?

如果说 PyTorch 是造车的发动机, 那 HuggingFace 就是直接送了你一辆跑车

你可以把HuggingFace看成:

- AI 界的 GitHub:

程序员找代码去 GitHub, AI 工程师找模型和数据去 HuggingFace。

它托管了全球最主流的开源模型 (Llama, BERT, Stable Diffusion 等) 。

- AI 界的 App Store:

它提供了很多现成的 Pipeline, 你不需要懂原理, 就能在你的代码里跑起来 (比如一键实现情感分析) 。

HuggingFace的核心

Thinking: HuggingFace的核心是什么?

我们在写代码时，主要和这三个库打交道：

1) Transformers (库):

工具箱。提供了统一的 Python 接口，可以用几乎相同的代码加载 BERT、GPT 或 Llama。

2) Model Hub (网站):

仓库。存放模型权重文件（.bin 或 .safetensors）和配置文件（config.json）。

3) Datasets (库):

燃料库。一键下载维基百科、医疗问答、电商评论等海量数据集，并自动处理成模型能读的格式。

HuggingFace中的概念 (Transformer)

Transformer 模型主要分为三个流派，功能截然不同：

1) Encoder-Only (编码器架构)

代表模型：BERT

能力：擅长理解和分析。

像一个阅读理解满分的学生。你给它一篇文章，它能告诉你文章的情感是正面的还是负面的，或者提取出文章里的人名。

应用：文本分类、实体识别、搜索匹配。

HuggingFace中的概念 (Transformer)

2) Decoder-Only (解码器架构)

代表模型：GPT系列, Llama, Qwen

能力：擅长生成。

像一个话唠小说家。你给它一个开头，它会根据概率不断猜下一个字是什么。

应用：聊天机器人、代码生成、创意写作。

3) Encoder-Decoder (编解码架构)

代表模型：T5, BART

能力：擅长转换。

像一个翻译官。听懂一句（Encode），然后重组为另一种语言说出来（Decode）。

应用：机器翻译、文本摘要。

HuggingFace中都有哪些常用的模型

NLP（自然语言处理）领域

- 基础理解类（Encoder-Only）：

BERT / RoBERTa：文本分类、命名实体识别（NER）的首选经典。

DistilBERT：BERT 的轻量化版本，推理速度更快，适合端侧应用。

- 生成类（Decoder-Only / 大语言模型）：

Llama 系列 (Meta)：目前最主流的开源大模型，适合对话、逻辑推理。

Qwen 系列 (阿里)：中文能力极强，开源社区的热门选择。

DeepSeek 系列：近期极具性价比的国产模型，常用于生成和思考任务。

GPT-2：虽然较老，但因其轻量，常作为教学文本生成的入门案例。

- 翻译与摘要（Encoder-Decoder）：

T5 / FLAN-T5：能够将所有 NLP 任务转换为“文本到文本”的格式。

BART：擅长文本纠错和长文档摘要。

HuggingFace中都有哪些常用的模型

CV（计算机视觉）与多模态领域

- ViT (Vision Transformer): 图像分类的标杆。
- Stable Diffusion: 目前最火的开源图像生成模型。
- CLIP (OpenAI): 连接文本与图像，常用于以图搜图或零样本分类。

HuggingFace中都有哪些常用Datasets

任务类型	常用数据集名称	简介
情感分析	IMDb / Amazon Polarity	5万条电影评论/3500万条商品评论，带正负标签。
问答系统	SQuAD / CoQA	斯坦福问答数据集，用于训练模型提取文章中的答案。
大模型预训练	FineWeb / C4	数千亿词规模的清洗后网页文本，是大模型进化的基石。
代码生成	MBPP / CodeContests	包含数千个编程问题及其正确答案。
语音识别	Common Voice	Mozilla 提供的多语言开源语音数据集。
中文特定	CLUECorpusSmall	常用中文通用语料库，适合初学者训练中文小模型。

<https://huggingface.co/datasets/stanfordnlp/imdb>

HuggingFace中的概念 (Tokenization)

Thinking: 什么是Tokenization?

模型不认识汉字或英文单词，它只认识数字。Tokenizer 是人类语言和机器数字之间的桥梁。

Input: "我爱HuggingFace"

Tokenize (切分): ['我', '爱', 'Hugging', '##Face'] (注: BERT常用WordPiece切分)

Convert to IDs (转数字): [2769, 4263, 12890, 8921]

Output: 模型接收的就是这串数字向量。

Tips: 不同的模型有专属的 Tokenizer, 不能混用! 用 BERT 的 Tokenizer 去处理数据喂给 GPT 模型, 会报错或输出乱码。

不同语言模型的Token都是如何定义的

Thinking: 不同语言模型的Token是如何定义的?

Token是大型语言模型处理文本的最小单位。由于模型本身无法直接理解文字，因此需要将文本切分成一个个Token，再将Token转换为数字（向量）进行运算。不同的模型使用不同的“分词器”（Tokenizer）来定义Token。

例如，对于英文 Hello World：

GPT-4o 会切分为 ["Hello", "World"] => 对应的 token id = [13225, 5922]

对于中文“人工智能你好啊”：

DeepSeek-R1会切分为 ["人工智能", "你好", "啊"] => 对应的token id = [33574, 30594, 3266]

分词方式的不同会直接影响模型的效率和对语言细节的理解能力。

可以通过 <https://tiktokenizer.vercel.app/> 这个工具看到不同模型是如何切分你输入的文本的。

模型的常见特殊Token

为了让模型更好地理解文本的结构和指令，开发者会预设一些具有特殊功能的Token。

这些Token不代表具体词义，而是作为一种“标点”或“命令”存在。

- 分隔符 (Separator Token):

用于区分不同的文本段落或角色。比如，在对话中区分用户和AI的发言，可能会用 `<|user|>` 和 `<|assistant|>` 这样的Token。

- 结束符 (End-of-Sentence/End-of-Text Token):

告知模型文本已经结束，可以停止生成了。常见的如 `[EOS]` 或 `<|endoftext|>`。这对于确保模型生成完整且不冗长的回答至关重要。

- 起始符 (Start Token):

标记序列的开始，例如 `[CLS]` (Classification) 或 `[BOS]` (Beginning of Sentence)，帮助模型准备开始处理文本。

HuggingFace使用环境（针对国内）

Thinking：按照官方文档写 `model.from_pretrained('bert-base-uncased')`，在国内大概率会报错 `ConnectionError`。这是为什么？

不是你代码写错了，是网络问题。

有两种最稳妥的解决方案。

方案 1：使用 HF-Mirror (镜像站)

不需要修改代码逻辑，只需要在运行 Python 之前设置一个环境变量，把流量指引到国内镜像站。

```
import os
# 设置环境变量，使用国内镜像站
os.environ['HF_ENDPOINT'] = 'https://hf-mirror.com'
from transformers import AutoModel, AutoTokenizer
# 此时会自动从镜像站下载，速度飞快
model = AutoModel.from_pretrained("bert-base-chinese")
```

在 Python 代码最开头（import transformers 之前）加入

HuggingFace使用环境（针对国内）

方案 2：使用 ModelScope (魔搭社区)

使用阿里的 ModelScope 下载文件，然后用 HuggingFace 加载。

=> ModelScope 是下载工具，HuggingFace 是加载工具。

```
# 1. 安装 modelscope: pip install modelscope
from modelscope import snapshot_download
from transformers import AutoModel, AutoTokenizer

# 2. 从 ModelScope 下载模型文件到本地
# 这里的 qwen/Qwen2.5-0.5B 是魔搭上的模型ID，通常和HF同名
model_dir = snapshot_download('qwen/Qwen2.5-0.5B')
print(f"模型已下载到: {model_dir}")
```

```
# 3. 使用 HuggingFace 原生库加载（注意：这里传入的是本地路径）
tokenizer = AutoTokenizer.from_pretrained(model_dir)
model = AutoModel.from_pretrained(model_dir)

# 4. 验证是否成功
print("模型加载成功！")
```

Pipeline与模型应用

Pipeline：一行代码实现AI功能

Thinking: Pipeline 是什么？

它是一个全自动的黑盒。当你把文本扔进去，它自动帮你完成三个繁琐的步骤：

- 预处理 (Tokenizer)：把文本变成数字。
- 模型推理 (Model)：模型进行计算，输出一堆看不懂的分數 (Logits) 。
- 后处理 (Post-processing)：把分数变成人类能看懂的标签 (比如 "Positive", 99%) 。

CASE1: 情感分析

TO DO: 使用huggingface进行情感分析

任务: 判断一条电商评论是好评还是差评。

```
import os
# 必须在导入 pipeline 之前设置
os.environ["HF_ENDPOINT"] = "https://hf-mirror.com"
from transformers import pipeline

# 1. 加载 pipeline
# 指定 task="sentiment-analysis"
# 技巧: 如果不指定 model, 默认下载英文模型。
# 这里我们指定一个中文微调过的模型, 效果更好。
cache_dir = '/root/autodl-tmp/models'
```

```
classifier = pipeline(
    task="sentiment-analysis",
    model="uer/roberta-base-finetuned-dianping-chinese",
    model_kwargs={"cache_dir": cache_dir}, # 模型权重缓存位置
    tokenizer_kwargs={"cache_dir": cache_dir}, # Tokenizer 缓存位置
)

# 2. 预测
result = classifier("这个手机屏幕太烂了, 反应很慢! ")
print(result)

result2 = classifier("物流很快, 包装很精美, 五星好评。")
print(result2)
```

```
[{'label': 'negative (stars 1, 2 and 3)', 'score': 0.9923227429389954}]
[{'label': 'positive (stars 4 and 5)', 'score': 0.9825959205627441}]
```

CASE2：文本生成

TO DO：文本生成

任务：给个开头，让 AI 帮你编故事。

```
import os
os.environ["HF_ENDPOINT"] = "https://hf-mirror.com"
os.environ["HF_HOME"] = "/root/autodl-tmp/models" # 同时控制 model 和 tokenizer 缓存
from transformers import pipeline

generator = pipeline(
    task="text-generation",
    model="uer/gpt2-chinese-cluecorpussmall",
    device="cuda"
)
```

```
text = "在一个风雨交加的夜晚，程序员小李打开了电脑，突然"
result = generator(text, max_length=100, truncation=True,
do_sample=True)
print(result[0]['generated_text'])
```

在一个风雨交加的夜晚，程序员小李打开了电脑，突然袭击了一条短信，说：大意是，程序员小李，想想自己以前也发过类似命令。但是你不要着急啊，看下你要点别的，他给你一个小招，让你改改命名！小李第一反应就是

2-文本生成.py

CASE3：零样本分类

TO DO：零样本分类

模型从来没见过你的标签，但它能理解其中的含义进行分类。

```
import os
# 必须在导入 pipeline 之前设置
os.environ["HF_ENDPOINT"] = "https://hf-mirror.com"
os.environ["HF_HOME"] = "/root/autodl-tmp/models"
from transformers import pipeline
# 加载零样本分类器 (推荐使用支持多语言的模型)
classifier = pipeline(
    task="zero-shot-classification",
    model="facebook/bart-large-mnli" # 这个模型虽然是英文
    的，但对简单中文也能理解
)
```

```
text = "特斯拉发布了最新的自动驾驶技术，股价大涨。"
candidate_labels = ["体育", "财经", "娱乐", "科技"]

# 让模型自己去匹配
result = classifier(text, candidate_labels)
print(f"文本内容：{text}")
print(f"预测标签：{result['labels'][0]} (置信度：
{result['scores'][0]:.4f})")
```

文本内容：特斯拉发布了最新的自动驾驶技术，股价大涨。

预测标签：科技 (置信度: 0.4399)

3-零样本分类.py

深入核心组件：Tokenizer

Thinking: Huggingface的Pipeline很好用，还要不要了解pipeline里面都封装了什么？还是直接用就可以了？

Huggingface是傻瓜相机。如果想要深度定制，或者微调模型，就要把 Pipeline 拆开

核心组件 1: Tokenizer (分词器)

作用：把文本变成 Tensor (张量)。

- 加载词表：必须和模型配套（不能用 BERT 的字典查 GPT 的词）。
- Padding (填充)：把短句子补长（通常补 0），因为 GPU 喜欢整齐的矩阵。
- Truncation (截断)：把太长的句子切掉，因为模型有最大长度限制（如 512）。

CASE4: tokenizer使用

TO DO: tokenizer使用

```
import os

# 必须在导入 pipeline 之前设置
os.environ["HF_ENDPOINT"] = "https://hf-mirror.com"
os.environ["HF_HOME"] = "/root/autodl-tmp/models"
from transformers import AutoTokenizer

# 加载分词器
tokenizer = AutoTokenizer.from_pretrained("bert-base-chinese")

# 模拟一个 Batch (两个长度不一样的句子)
sentences = ["我爱AI", "HuggingFace真好用"]
```

重点观察: input_ids: 你的字对应的数字

attention_mask: 1代表是真实的字, 0代表是填充的(Padding)

调用分词器

```
inputs = tokenizer(
    sentences,
    padding=True,    # 自动填充到最长句子的长度
    truncation=True, # 超过最大长度就截断
    max_length=10,   # 设置最大长度
    return_tensors="pt" # 返回 PyTorch 张量
)

print(inputs)
```

```
{'input_ids': tensor([[ 101, 2769, 4263, 100, 102,  0],
                      [ 101, 100, 4696, 1962, 4500, 102]]), 'token_type_ids': tensor([[0, 0, 0, 0, 0, 0],
                      [0, 0, 0, 0, 0, 0]]), 'attention_mask': tensor([[1, 1, 1, 1, 1, 0],
                      [1, 1, 1, 1, 1, 1]])}
```

深入核心组件：Model

Thinking: 在Huggingface中经常会看到 `AutoModel` vs `AutoModelFor...` 这两个分别是什么？

`AutoModel` (Base Model): 只有大脑。它输出的是隐藏状态 (Hidden States), 一堆高维向量

=> 人类看不懂, 适合做从头搭建网络。

`AutoModelForSequenceClassification` (带头模型): 大脑 + 嘴巴。

=> 它在 Base Model 后面加了一个全连接层 (Classification Head), 直接输出分类的分数。

CASE: Model使用

TO DO: Model使用

```
import os

# 必须在导入 pipeline 之前设置
os.environ["HF_ENDPOINT"] = "https://hf-mirror.com"
os.environ["HF_HOME"] = "/root/autodl-tmp/models" # 同时控制 model 和 tokenizer 缓存

from transformers import AutoModel,
AutoModelForSequenceClassification

# 1. 纯净版模型
base_model = AutoModel.from_pretrained("bert-base-chinese")
base_model
```

```
BertModel(
  (embeddings): BertEmbeddings(
    (word_embeddings): Embedding(21128, 768, padding_idx=0)
    (position_embeddings): Embedding(512, 768)
    (token_type_embeddings): Embedding(2, 768)
    (LayerNorm): LayerNorm((768,), eps=1e-12, elementwise_affine=True)
    (dropout): Dropout(p=0.1, inplace=False)
  )
  (encoder): BertEncoder(
    (layer): ModuleList(
      (0-11): 12 x BertLayer(
        (attention): BertAttention(
          (self): BertSdpaSelfAttention(
            (query): Linear(in_features=768, out_features=768, bias=True)
            (key): Linear(in_features=768, out_features=768, bias=True)
            (value): Linear(in_features=768, out_features=768, bias=True)
            (dropout): Dropout(p=0.1, inplace=False)
          )
          (output): BertSelfOutput( ...
            ....
          )
        )
      )
    )
    (pooler): BertPooler(
      (dense): Linear(in_features=768, out_features=768, bias=True)
      (activation): Tanh()
    )
  )
)
```

CASE: Model使用

2. 带分类头的模型

```
cls_model =
```

```
AutoModelForSequenceClassification.from_pretrained("bert-  
base-chinese", num_labels=2)
```

```
# 输出形状: (Batch_Size, Num_Labels) -> (2, 2)
```

```
cls_model
```

```
BertForSequenceClassification(  
  (bert): BertModel(  
    (embeddings): BertEmbeddings(  
      (word_embeddings): Embedding(21128, 768, padding_idx=0)  
      (position_embeddings): Embedding(512, 768)  
      (token_type_embeddings): Embedding(2, 768)  
      (LayerNorm): LayerNorm((768,), eps=1e-12, elementwise_affine=True)  
      (dropout): Dropout(p=0.1, inplace=False)  
    )  
    (encoder): BertEncoder(  
      (layer): ModuleList(  
        (0-11): 12 x BertLayer(  
          (attention): BertAttention(  
            (self): BertSdpaSelfAttention(  
              (query): Linear(in_features=768, out_features=768, bias=True)  
              (key): Linear(in_features=768, out_features=768, bias=True)  
            )  
            ...  
          )  
          (pooler): BertPooler(  
            (dense): Linear(in_features=768, out_features=768, bias=True)  
            (activation): Tanh()  
          )  
        )  
      )  
      (dropout): Dropout(p=0.1, inplace=False)  
      (classifier): Linear(in_features=768, out_features=2, bias=True)  
    )  
  )  
)
```


CASE：手写从输入文本到获得概率的全过程

TO DO：手写从输入文本到获得概率的全过程

```
import os

# 必须在导入 pipeline 之前设置
os.environ["HF_ENDPOINT"] = "https://hf-mirror.com"
os.environ["HF_HOME"] = "/root/autodl-tmp/models"
import torch

import torch.nn.functional as F

from transformers import AutoTokenizer,
AutoModelForSequenceClassification

# 1. 准备工作
checkpoint = "uer/roberta-base-finetuned-dianping-chinese"
tokenizer = AutoTokenizer.from_pretrained(checkpoint)
model =
AutoModelForSequenceClassification.from_pretrained(checkpoint)
```

2. 原始文本

```
text = "这家餐厅太难吃了，服务员态度还差！"
```

3. Step 1: Tokenize (预处理)

```
inputs = tokenizer(text, return_tensors="pt")
```

```
# inputs 包含 input_ids 和 attention_mask
```

4. Step 2: Model Inference (模型推理)

```
# 不需要计算梯度，节省内存
```

```
with torch.no_grad():
```

```
    outputs = model(**inputs)
```

```
    # 获取 Logits (未归一化的分数)
```

```
    logits = outputs.logits
```

```
    print(f"模型原始输出 (Logits): {logits}")
```

CASE：手写从输入文本到获得概率的全过程

```
# 5. Step 3: Post-processing (后处理)
# 使用 Softmax 将 logits 转换为概率
probabilities = F.softmax(logits, dim=-1)
print(f"概率分布: {probabilities}")

# 获取概率最大的标签索引
predicted_id = torch.argmax(probabilities).item()
# 使用 model.config.id2label 查表得到人类可读标签
predicted_label = model.config.id2label[predicted_id]

print(f"最终结果: {predicted_label}")
```

模型原始输出 (Logits): tensor([[2.8590, -2.7449]])

概率分布: tensor([[0.9963, 0.0037]])

最终结果: negative (stars 1, 2 and 3)

Pipeline：常用任务（Task）速查

pipeline 是 HuggingFace 提供的最简易接口，只需指定 task 名称即可直接调用。

- NLP 常用任务

sentiment-analysis：情感分析，判断文本褒贬。

text-generation：文本生成，如写故事、续写代码。

zero-shot-classification：零样本分类，无需训练即可根据自定义标签分类。

question-answering：问答系统，根据给定的上下文回答问题。

summarization：自动摘要，将长文浓缩为短句。

translation_xx_to_yy：翻译任务，如 translation_en_to_zh。

ner：命名实体识别，提取人名、地名、组织名。

Pipeline：常用任务（Task）速查

- CV & 音频 常用任务

image-classification：图像分类。

object-detection：目标检测，识别图中的物体并定位。

automatic-speech-recognition (ASR)：语音转文字。

text-to-speech (TTS)：文字转语音。

Summary

- 模型不仅仅是代码，它是一种能力；
- 数据集不仅仅是文件，它是知识；
- 而 Pipeline 则是将能力与知识打包好的一键服务。

Pipeline 是最快的落地方式，Demo 阶段首选。

- Tokenizer 的 padding 和 truncation 是批处理数据的关键。
- AutoModelFor... 系列封装了特定任务的头，我们在微调时通常使用这一类模型。

全量微调

为什么需要微调 (Fine-tuning)

Thinking: HuggingFace 上模型那么多, 我直接拿来用不行吗? 为什么非要自己训练?

Pre-training (预训练) = 大学通识教育。

模型 (比如 BERT) 读了海量的维基百科、书籍。

能力: 它懂语法, 懂成语, 知道“苹果”既是水果也是公司。

局限: 它不知道你公司具体的业务逻辑。

Fine-tuning (微调) = 岗前专业培训。

动作: 在预训练模型的基础上, 用你特定领域的数据再训练一小会儿。

目标: 让它从“懂中文的毕业生”变成“懂医疗发票的审核员”。

应用场景

场景举例：通用模型搞不定的事

- **通用 BERT**：看到“CT显示肺部纹理增多”，它知道这是一句话。
- **医疗微调 BERT**：能精准识别出 CT (检查项), 肺部 (解剖部位), 纹理增多 (临床表现)。
- **垃圾邮件场景**：通用模型可能觉得“恭喜您中奖了”是句好话 (Positive) ,
但在我们的业务里，它是 100% 的垃圾邮件 (Spam) 。

核心工具： Datasets 与 DataCollator

1. 数据加载与清洗 (Datasets)

在实际工作中，数据通常不在 Hub 上，而在你的 CSV 或 Excel 里。

- 加载： `load_dataset("csv", data_files="my_data.csv")`
- Map 函数 (神器):

作用：它不是简单的 for 循环，而是支持多进程的并行处理。

操作：我们需要把文本列 (Text) 批量转换成数字列 (Input IDs) 。

核心工具：Datasets 与 DataCollator

2. DataCollator：动态补齐

- 痛点：BERT 模型要求同一批（Batch）进来的数据长度必须一致。
- 传统做法：

不管句子多长，全部补齐到 512。

缺点：如果一句话只有 10 个字，你补了 502 个 0，显卡在疯狂计算无效的 0，浪费显存和时间。

- 聪明做法 (动态补齐)：

在训练时，看这一批数据（比如 16 条）里最长的那句是多少（比如 50）。

把其他 15 条都只补齐到 50。

结果：训练速度极大提升，显存占用大幅下降。

训练神器：Trainer API

Thinking：使用Trainer API的好处是什么？

在 HuggingFace 出现之前，写一个训练循环需要手动写出所有步骤。

你要自己写 Forward, Backward, Optimizer.step, Zero_grad... 写错一步模型就不收敛。

现在，Trainer 把这些脏活累活全包了。

维度	传统 PyTorch 写法	HuggingFace Trainer
代码量	50+ 行循环代码	实例化 1 个类
功能支持	需手写日志、保存、断点续训	开箱即用 (Checkpoints, Logging)
硬件优化	需手动配置混合精度 (fp16)	参数 fp16=True 即可
多卡训练	配置 DistributedDataParallel (极难)	自动适配多卡

CASE：垃圾邮件分类器

TO DO：搭建垃圾邮件分类器

垃圾邮件分类是经典二分类任务，用户每天都会收到大量的邮件和短信，其中不乏垃圾信息（如诈骗信息、广告推广、恶意链接等）。手动筛选这些信息不仅耗时耗力，而且容易遗漏。

使用 HuggingFace 的 Transformers 库，基于预训练的 BERT 中文模型（bert-base-chinese）进行微调，构建一个能够自动识别垃圾邮件的分类器。

输入：一条短信或邮件文本。

输出：0 (正常), 1 (垃圾)。

模型：bert-base-chinese (中文小模型，显存友好)。

CASE：垃圾邮件分类器

Step1, 准备环境与数据

首先配置 HuggingFace 镜像站，确保在国内环境下能够顺利下载模型

```
import os  
os.environ["HF_ENDPOINT"] = "https://hf-mirror.com"  
os.environ["HF_HOME"] = "/root/autodl-tmp/models"
```

准备训练数据，每条数据包含文本内容和标签（0表示正常，1表示垃圾）：

```
data = [  
    {"text": "今晚有空一起吃饭吗？ ", "label": 0}, # 正常  
    {"text": "恭喜您获得500万大奖，点击领取", "label": 1}, # 垃圾  
    # ... 更多数据  
]  
  
dataset = Dataset.from_list(data)  
dataset = dataset.train_test_split(test_size=0.2) # 划分训练集和测试集
```

CASE：垃圾邮件分类器

Step2, 数据预处理 (Tokenization)

使用 BERT 的 Tokenizer 将文本转换为模型可理解的数字序列。这里的关键是不进行 Padding, 留给 DataCollator 动态处理

```
checkpoint = "bert-base-chinese"
tokenizer = AutoTokenizer.from_pretrained(checkpoint)

def preprocess_function(examples):
    return tokenizer(examples["text"], truncation=True, max_length=128)

tokenized_datasets = dataset.map(preprocess_function, batched=True)
```

truncation=True: 截断过长的文本 (BERT 最大长度限制)

padding=False: 不在此处补齐, 避免浪费显存

CASE：垃圾邮件分类器

Step3, DataCollator 动态补齐

传统做法是将所有样本补齐到固定长度（如512），这会浪费大量显存。

动态补齐，只补齐到当前批次中最长的长度，大幅提升训练效率

```
from transformers import DataCollatorWithPadding  
data_collator = DataCollatorWithPadding(tokenizer=tokenizer)
```

Step4, 模型加载

加载带分类头的 BERT 模型，num_labels=2 表示二分类任务

```
model = AutoModelForSequenceClassification.from_pretrained(  
    checkpoint,  
    num_labels=2  
)
```

AutoModelForSequenceClassification 在基础 BERT 模型后添加了分类头，可以直接输出分类结果。

CASE：垃圾邮件分类器

Step5，评估指标定义

定义准确率作为评估指标

```
import numpy as np
import evaluate

metric = evaluate.load("accuracy")

def compute_metrics(eval_pred):
    logits, labels = eval_pred
    predictions = np.argmax(logits, axis=-1)
    return metric.compute(predictions=predictions, references=labels)
```


CASE：垃圾邮件分类器

Step6, 训练配置与执行

使用 Trainer API 简化训练流程，只需配置参数即可

```
training_args = TrainingArguments(  
    output_dir="./spam-bert-finetuned",  
    eval_strategy="epoch",          # 每个 epoch 结束后评估  
    save_strategy="epoch",          # 每个 epoch 结束后保存  
    learning_rate=2e-5,             # 微调学习率通常很小  
    per_device_train_batch_size=4,  
    num_train_epochs=3,  
    weight_decay=0.01,  
    logging_steps=10,  
)
```

```
trainer = Trainer(  
    model=model,  
    args=training_args,  
    train_dataset=tokenized_datasets["train"],  
    eval_dataset=tokenized_datasets["test"],  
    tokenizer=tokenizer,  
    data_collator=data_collator,  
    compute_metrics=compute_metrics,  
)  
trainer.train()
```

Trainer API 的优势：自动处理训练循环（Forward、Backward、Optimizer）

支持混合精度训练、多卡训练；自动保存检查点、记录日志

CASE：垃圾邮件分类器

Step7, 模型推理

训练完成后，使用模型进行预测

```
text = "低息贷款，无抵押，秒到账"
inputs = tokenizer(text, return_tensors="pt").to(model.device)

with torch.no_grad():
    logits = model(**inputs).logits
    predicted_class_id = logits.argmax().item()

print(f"预测类别: {'垃圾邮件' if predicted_class_id == 1 else '正常邮件'})
```

Summary

- 微调 vs 预训练:

预训练模型已经具备语言理解能力，微调只需要少量领域数据即可适应特定任务

- 动态补齐:

使用 DataCollatorWithPadding 实现批次级别的动态补齐，提升训练效率

- Trainer API:

封装了训练的所有细节，让开发者专注于数据和模型本身

- 评估策略:

每个 epoch 结束后进行评估和保存，便于监控训练过程

HuggingFace 生态简化 NLP 任务的开发流程，从数据准备到模型训练，再到最终推理，整个过程清晰高效。

CASE：垃圾邮件分类器-Qwen

TO DO：在Huggingface中使用qwen2.5-7b-instruct 大模型进行垃圾邮件分类

使用大语言模型（Qwen），通过精心设计的 Prompt 让模型理解任务并生成分类结果，无需训练。

针对国内的环境，模型下载可以使用 modelscope

```
from modelscope import snapshot_download
from transformers import AutoModelForCausalLM, AutoTokenizer

# Step 1: 使用 ModelScope 下载模型（国内速度快）
modelscope_model_id = "Qwen/Qwen2.5-7B-Instruct"
cache_dir = "/root/autodl-tmp/models"

model_dir = snapshot_download(
    modelscope_model_id,
    cache_dir=cache_dir
)
```

8-垃圾邮件分类器-Qwen.py

CASE：垃圾邮件分类器-Qwen

```
# Step 2: 使用 HuggingFace 加载本地模型
tokenizer = AutoTokenizer.from_pretrained(model_dir,
trust_remote_code=True)
model = AutoModelForCausalLM.from_pretrained(
    model_dir,
    torch_dtype=torch.float16, # 半精度节省显存
    device_map="auto", # 自动分配设备（支持多卡）
    trust_remote_code=True
)
```

- ModelScope 是下载工具：负责从魔搭社区下载模型文件
- HuggingFace 是加载工具：使用统一的 Transformers API 加载和使用
- AutoModelForCausalLM：因果语言模型，用于文本生成
- device_map="auto"：自动分配设备，支持多 GPU

CASE：垃圾邮件分类器-Qwen

Prompt 设计

```
def classify_spam_with_prompt(text, model, tokenizer):  
    # 构建分类提示词  
    prompt = f'""你是一个垃圾邮件分类专家。请判断以下文  
本是否为垃圾邮件。  
文本：{text}  
请只回答"垃圾邮件"或"正常邮件": ""'  
    # Tokenize  
    inputs = tokenizer(prompt, return_tensors="pt")  
    if torch.cuda.is_available():  
        inputs = inputs.to(model.device)  
    # 生成回答  
    with torch.no_grad():  
        outputs = model.generate(  
            **inputs,
```

```
            max_new_tokens=15, # 只需要生成少量 token  
            do_sample=False, # 贪心解码，保证稳定  
            pad_token_id=tokenizer.eos_token_id,  
        )  
    # 解码输出  
    generated_text = tokenizer.decode(  
        outputs[0][inputs['input_ids'].shape[1]:],  
        skip_special_tokens=True  
    ).strip()
```

```
    # 解析结果  
    if "垃圾" in generated_text:  
        return 1, "垃圾邮件"  
    elif "正常" in generated_text:  
        return 0, "正常邮件"
```

**model.generate()：生成文本，
而不是直接分类**

8-垃圾邮件分类器-Qwen.py

CASE：垃圾邮件分类器-Qwen

Prompt 设计

```
from transformers import pipeline

# 创建 Pipeline
generator = pipeline(
    "text-generation",
    model=model,
    tokenizer=tokenizer
    # 不指定 device, Pipeline 会自动检测模型所在设备
)

# 使用 Pipeline 分类
def classify_spam_with_pipeline(text, generator):
    prompt = f"""你是一个垃圾邮件分类专家。请判断以下文本是否为垃圾邮件。
    文本：{text}
    请只回答"垃圾邮件"或"正常邮件": """
```

```
result = generator(
    prompt,
    max_new_tokens=15,
    do_sample=False,
    return_full_text=False, # 只返回新生成的部分
)

generated_text = result[0]['generated_text'].strip()
# 解析结果...
```

Pipeline 封装了 tokenize、generate、decode 流程
代码更简洁，适合快速原型开发

8-垃圾邮件分类器-Qwen.py

Summary

- BERT 微调方式:

文本 → Tokenize → 模型前向传播 → Logits → argmax → 类别ID

- 大模型 Prompt 方式:

文本 → 构建 Prompt → Tokenize → 模型生成 → 解码文本 → 解析结果 → 类别

Thinking: 什么时候用BERT微调, 什么时候用大模型?

BERT: 有大量标注数据, 任务固定, 需要高准确率, 快速推理 (生产环境), 显存有限 (1-2GB)

大模型: 没有标注数据, 任务多样, 需要快速验证, 需要零样本学习能力, 有足够的显存 (8GB+)

打卡：搭建垃圾邮件分类器



使用BERT微调，搭建一个垃圾邮件分类器，掌握Huggingface模型微调的步骤

Step1, 准备环境与数据

Step2, 数据预处理 (Tokenization)

Step3, DataCollator 动态补齐

Step4, 模型加载

Step5, 评估指标定义

Step6, 训练配置与执行

Step7, 模型推理



Thank You
Using data to solve problems